

Assignment No. 06

- **TITLE** – Design a menu-driven program to implement and compare different search techniques Linear, Sentinel, Binary, and Fibonacci Search on sorted and unsorted datasets. Count and display the number of comparisons made by each algorithm to evaluate their efficiency.

➤ **THEORY** –

Search algorithms are fundamental operations in computer science used to locate specific elements within datasets. The efficiency of these algorithms varies significantly based on:

- **Data organization** (sorted/unsorted)
- **Algorithm design** (sequential/divid-and-conquer)
- **Time complexity** (linear/logarithmic)

Key Search Techniques:

- **Linear & Sentinel Search:** Suitable for unsorted data, $O(n)$ complexity
- **Binary & Fibonacci Search:** Require sorted data, $O(\log n)$ complexity

➤ **ALGORITHM** -

1. Linear Search

Step 1: Start from the first element

Step 2: Compare each element with target sequentially

Step 3: If match found, return index

Step 4: If end reached, return -1

Time Complexity: $O(n)$

2. Sentinel Search

Step 1: Replace last element with target

Step 2: Search sequentially without bounds checking

Step 3: If element found before last position, return index

Step 4: If last position contains target, return index

Step 5: Restore original last element

Time Complexity: $O(n)$

3. Binary Search

Step 1: Set left=0, right=n-1

Step 2: While left <= right:

mid = (left + right) / 2

Step 3: If arr[mid] == target, return mid

Step 4: If arr[mid] < target, left = mid + 1

Step 5: Else right = mid - 1

Step 6: Return -1 if not found

Time Complexity: O(log n)

4. Fibonacci Search

Step 1: Initialize Fibonacci numbers: fib2=0, fib1=1, fib=1

Step 2: Find smallest Fibonacci number $\geq n$

Step 3: While fib > 1:

i = min(offset + fib2, n-1)

Step 4: If arr[i] < target:

Move Fibonacci sequence forward

offset = i

Step 5: Else if arr[i] > target:

Move Fibonacci sequence backward

Step 6: Else return i

Step 7: Return -1 if not found

Time Complexity: O(log n)

Program Flow

1. Input array size and choose sorted/unsorted data
2. Enter target element to search
3. Based on data type, offer appropriate search options:
 - Unsorted: Linear, Sentinel
 - Sorted: Binary, Fibonacci
4. Display search result with comparison count
5. Analyze time complexity case (Best/Average/Worst)

➤ **CODE –**

1. search_techniques.cpp

```
#include<iostream>
#include<algorithm>
#include<cmath>
using namespace std;

int comparisons = 0;
void reset(){
    comparisons = 0;
}

int linear(int arr[], int n, int target){
    reset();
    for(int i = 0;i<n;i++){
        comparisons++;
        if(arr[i]==target){
            return i;
        }
    }
    return -1;
}

int binary(int arr[],int n,int target){
    reset();
    int left = 0;
    int right = n-1;
    while(left<=right){
        int mid = left + (right-left)/2;
        if(arr[mid]==target){
            comparisons++;
            return mid;
        }
        else if(arr[mid]<target){
            comparisons++;
            left = mid + 1;
        }
        else{
            right = mid - 1;
            comparisons++;
        }
    }
    return -1;
}

int sentinel(int arr[],int n, int target){
    reset();
    int last = arr[n-1];
    arr[n-1] = target;
    int i = 0;
    while(arr[i]!=target){
        i++;
    }
}
```

```

    arr[n-1] = last;
    if(i<n-1 || arr[n-1]==target){
        comparisons++;
        return i;
    }
}

int fibonacci(int arr[], int n, int target){
    reset();
    int fib2 = 0;
    int fib1 = 1;
    int fib = fib1 + fib2;
    int offset = -1;
    while(fib<n){
        fib2 = fib1;
        fib1 = fib;
        fib = fib1 + fib2;
    }
    while(fib>1){
        int i = min(offset+fib2, n-1);
        if(arr[i]<target){
            fib = fib1;
            fib1 = fib2;
            fib2 = fib - fib1;
            offset = i;
            comparisons++;
        }
        else if(arr[i]>target){
            fib = fib2;
            fib1 = fib1 - fib2;
            fib2 = fib - fib1;
            comparisons++;
        }
        else{
            comparisons++;
            return i;
        }
    }
}

int main(){
    int n, m, target, ch1, ch2, result = 0, c = 0;
    cout<<"Enter the size of array: "<<endl;
    cin>>n;
    int arr[n];
    cout<<"Choose any one!\n 1. Create Sorted Array \n 2. Create Unsorted Array \n 3.
Exit"<<endl;
    cin>>ch1;
    switch(ch1){
        case 1:
            cout<<"Enter elements in sorted way: "<<endl;
            for(int i = 0;i<n;i++)

```

```

        cin>>arr[i];
    break;

    case 2:
    cout<<"Enter elements in unsorted way: "<<endl;
    for(int i = 0;i<n;i++)
        cin>>arr[i];
    break;

    case 3:
    cout<<"Exitting..."<<endl;
    return 0;

    default:
    cout<<"Invalid Option!"<<endl;
}
cout<<"Enter the target element: "<<endl;
cin>>target;
while(true){
    if(ch1==2){
        cout<<"Choose any one!\n 1. Linear Search\n 2. Sentinel Search \n 5.
Exit"<<endl;
        cin>>ch2;
        switch(ch2){
            case 1:
            result = linear(arr,n,target);
            cout<<"Element found at index: "<<result<<endl;
            c = comparisons;
            cout<<"Number of comparisons: "<<c<<endl;
            if(c==1){
                cout<<"Best Case - O(1)"<<endl;
            }
            else if(c==n){
                cout<<"Worst Case - O(n)"<<endl;
            }
            else{
                cout<<"Average Case - O(n)"<<endl;
            }
            break;

            case 2:
            result = sentinel(arr,n,target);
            cout<<"Element found at index: "<<result<<endl;
            c = comparisons;
            cout<<"Number of comparisons: "<<c<<endl;
            if(c==1){
                cout<<"Best Case - O(1)"<<endl;
            }
            else if(c==n){
                cout<<"Worst Case - O(n)"<<endl;
            }
            else{

```

```

        cout<<"Average Case - O(n)"<<endl;
    }
    break;

    case 5:
    cout<<"Exiting..."<<endl;
    return 0;

    default:
    cout<<"Invalid choice!"<<endl;
}
}
if(ch1==1){
    cout<<"Choose any one!\n 3. Binary Search \n 4. Fibonacci Search\n 5.
Exit"<<endl;
    cin>>ch2;
    switch(ch2){

        case 3:
        result = binary(arr,n,target);
        cout<<"Element found at index: "<<result<<endl;
        c = comparisons;
        cout<<"Number of comparisons: "<<c<<endl;
        if(c==1){
            cout<<"Best Case - O(1)"<<endl;
        }
        else if(c==log(n)){
            cout<<"Worst Case - O(log n)"<<endl;
        }
        else{
            cout<<"Average Case - O(log n)"<<endl;
        }
        break;

        case 4:
        result = fibonacci(arr,n,target);
        cout<<"Element found at index: "<<result<<endl;
        c = comparisons;
        cout<<"Number of comparisons = "<<c<<endl;
        if(c==1){
            cout<<"Best Case - O(1)"<<endl;
        }
        else if(c==log(n)){
            cout<<"Worst Case - O(log n)"<<endl;
        }
        else{
            cout<<"Average Case - O(log n)"<<endl;
        }
        break;

        case 5:
        cout<<"Exiting..."<<endl;
    }
}
}

```

```

return 0;

default:
    cout<<"Invalid choice!"<<endl;
}
}
}
return 0;
}

```

➤ OUTPUT –

For Sorted arrays -

```

Enter the size of array:
6
Choose any one!
1. Create Sorted Array
2. Create Unsorted Array
3. Exit
1
Enter elements in sorted way:
12 23 34 45 56 67
Enter the target element:
56
Choose any one!
3. Binary Search
4. Fibonacci Search
5. Exit
3
Element found at index: 4
Number of comparisons: 2
Average Case -  $O(\log n)$ 
Choose any one!
3. Binary Search
4. Fibonacci Search
5. Exit
4
Element found at index: 4
Number of comparisons = 2
Average Case -  $O(\log n)$ 

```

For unsorted arrays –

Enter the size of array: 7	Enter the target element: 8	Choose any one!
Choose any one!	Choose any one!	1. Linear Search
1. Create Sorted Array	1. Linear Search	2. Sentinel Search
2. Create Unsorted Array	2. Sentinel Search	5. Exit
3. Exit	5. Exit	2
2	1	Element found at index: 4
Enter elements in unsorted way:	Element found at index: 4	Number of comparisons: 1
1 7 6 5 8 9 2	Number of comparisons: 5	Best Case - $O(1)$
	Average Case - $O(n)$	

➤ CONCLUSION –

This practical lab demonstrates how data organization dramatically impacts search efficiency, with sorted data enabling much faster logarithmic search algorithms.